

My Project

1

Generated by Doxygen 1.9.7

1 Namespace Index	1
1.1 Namespace List	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Namespace Documentation	7
4.1 MLWENO Namespace Reference	7
4.1.1 Detailed Description	7
4.1.2 Enumeration Type Documentation	7
4.1.2.1 stencilType	7
5 Class Documentation	9
5.1 MLWENO::basisPolynomial Class Reference	9
5.1.1 Constructor & Destructor Documentation	9
5.1.1.1 basisPolynomial() [1/3]	9
5.1.1.2 basisPolynomial() [2/3]	10
5.1.1.3 basisPolynomial() [3/3]	10
5.1.1.4 ~basisPolynomial()	10
5.1.2 Member Function Documentation	10
5.1.2.1 der() [1/2]	10
5.1.2.2 der() [2/2]	10
5.1.2.3 eval() [1/2]	10
5.1.2.4 eval() [2/2]	11
5.1.2.5 getCoef() [1/2]	11
5.1.2.6 getCoef() [2/2]	11
5.1.2.7 getI()	11
5.1.2.8 getJ()	11
5.1.2.9 getSize()	11
5.1.2.10 operator>() [1/2]	11
5.1.2.11 operator>() [2/2]	12
5.1.2.12 printCoef()	12
5.1.2.13 setCoef()	12
5.1.2.14 setMaxDegree()	12
5.2 MeshInfo Struct Reference	12
5.2.1 Member Data Documentation	13
5.2.1.1 cellGhostLayerSize	13
5.2.1.2 dim	13
5.2.1.3 edgeCorner	13
5.2.1.4 faceCorner	13
5.2.1.5 lmesh	13

5.2.1.6 localVals	13
5.2.1.7 MPIglobalCellSize	13
5.2.1.8 MPIglobalHoriEdgeSize	14
5.2.1.9 MPIglobalVertEdgeSize	14
5.2.1.10 MPIglobalVertexSize	14
5.2.1.11 MPIlocalCellSize	14
5.2.1.12 MPIlocalCellSizeFull	14
5.2.1.13 MPIlocalCellStart	14
5.2.1.14 MPIlocalHoriEdgeSize	14
5.2.1.15 MPIlocalVertEdgeSize	14
5.2.1.16 MPIlocalVertexSize	15
5.2.1.17 MPIlocalVertexSizeFull	15
5.2.1.18 MPIlocalVertexStart	15
5.2.1.19 vertexGhostLayerSize	15
5.2.1.20 volumeCorner	15
5.3 MLWENO::multiLevelReconstruction Class Reference	15
5.3.1 Constructor & Destructor Documentation	16
5.3.1.1 multiLevelReconstruction() [1/4]	16
5.3.1.2 multiLevelReconstruction() [2/4]	16
5.3.1.3 multiLevelReconstruction() [3/4]	16
5.3.1.4 multiLevelReconstruction() [4/4]	17
5.3.1.5 ~multiLevelReconstruction()	17
5.3.2 Member Function Documentation	17
5.3.2.1 AddLevel() [1/3]	17
5.3.2.2 AddLevel() [2/3]	17
5.3.2.3 AddLevel() [3/3]	17
5.3.2.4 AddReconstMethod() [1/2]	18
5.3.2.5 AddReconstMethod() [2/2]	18
5.3.2.6 Clear()	18
5.3.2.7 GetInfo()	18
5.3.2.8 PrintNonLinearWgts()	18
5.3.2.9 PrintSmoothnessIndicator()	18
5.3.2.10 UpdateNonLinearWgts()	19
5.3.2.11 UpdateTwoStageNonLinearWgts()	19
5.4 MLWENO::singleLevelReconstruction Class Reference	19
5.4.1 Constructor & Destructor Documentation	20
5.4.1.1 singleLevelReconstruction() [1/2]	20
5.4.1.2 singleLevelReconstruction() [2/2]	20
5.4.1.3 ~singleLevelReconstruction()	20
5.4.2 Member Function Documentation	20
5.4.2.1 CalculateSmoothnessIndic()	20
5.4.2.2 CheckExist()	21

5.4.2.3 CheckStencilPolynomials()	21
5.4.2.4 CheckStencils()	21
5.4.2.5 CreateStencilPolynomials()	21
5.4.2.6 GetScale() [1/2]	21
5.4.2.7 GetScale() [2/2]	21
5.4.2.8 GetSizeX()	22
5.4.2.9 GetSizeY()	22
5.4.2.10 PrintSmoothnessIndicator()	22
5.5 MLWENO::stencil< T > Class Template Reference	22
5.5.1 Constructor & Destructor Documentation	23
5.5.1.1 stencil() [1/4]	23
5.5.1.2 stencil() [2/4]	23
5.5.1.3 stencil() [3/4]	23
5.5.1.4 stencil() [4/4]	23
5.5.1.5 ~stencil()	23
5.5.2 Member Function Documentation	23
5.5.2.1 clearPtr()	24
5.5.2.2 CreateStencil()	24
5.5.2.3 getI()	24
5.5.2.4 getJ()	24
5.5.2.5 getK()	24
5.5.2.6 getSize()	24
5.5.2.7 GetStencil()	24
5.5.2.8 operator()() [1/6]	25
5.5.2.9 operator()() [2/6]	25
5.5.2.10 operator()() [3/6]	25
5.5.2.11 operator()() [4/6]	25
5.5.2.12 operator()() [5/6]	25
5.5.2.13 operator()() [6/6]	25
5.5.2.14 setI()	26
5.5.2.15 setJ()	26
5.5.2.16 setK()	26
5.5.2.17 SetStencil() [1/3]	26
5.5.2.18 SetStencil() [2/3]	26
5.5.2.19 SetStencil() [3/3]	26
5.6 MLWENO::stencilPolynomial Class Reference	27
5.6.1 Constructor & Destructor Documentation	27
5.6.1.1 stencilPolynomial() [1/3]	27
5.6.1.2 stencilPolynomial() [2/3]	27
5.6.1.3 stencilPolynomial() [3/3]	27
5.6.1.4 ~stencilPolynomial()	28
5.6.2 Member Function Documentation	28

5.6.2.1 eval() [1/2]	28
5.6.2.2 eval() [2/2]	28
5.6.2.3 GetOrderX()	28
5.6.2.4 GetOrderY()	28
5.6.2.5 GetScale()	28
5.6.2.6 GetSmoothIndic()	28
5.6.2.7 operator() [1/2]	29
5.6.2.8 operator() [2/2]	29
5.6.2.9 printCoef() [1/2]	29
5.6.2.10 printCoef() [2/2]	29
5.6.2.11 SetCollapsePolyn()	29
5.6.2.12 SetStencilPolynomials()	29
5.6.2.13 SetUpScale() [1/2]	30
5.6.2.14 SetUpScale() [2/2]	30
6 File Documentation	31
6.1 /home/renpo/Research/MantleSolver/src/mlweno/mlreconstruction.cpp File Reference	31
6.2 /home/renpo/Research/MantleSolver/src/mlweno/mlreconstruction.h File Reference	31
6.3 mlreconstruction.h	32
6.4 /home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp File Reference	32
6.4.1 Function Documentation	33
6.4.1.1 basePoly()	33
6.5 /home/renpo/Research/MantleSolver/src/mlweno/polynomial.h File Reference	33
6.6 polynomial.h	33
6.7 /home/renpo/Research/MantleSolver/src/mlweno/slireconstruction.cpp File Reference	35
6.8 /home/renpo/Research/MantleSolver/src/mlweno/slireconstruction.h File Reference	35
6.9 slreconstruction.h	35
6.10 /home/renpo/Research/MantleSolver/src/mlweno/stencil.h File Reference	36
6.11 stencil.h	37
6.12 /home/renpo/Research/MantleSolver/src/mlweno/util.cpp File Reference	38
6.12.1 Function Documentation	38
6.12.1.1 AssignValuesMeshInfo()	38
6.12.1.2 Bend() [1/2]	39
6.12.1.3 Bend() [2/2]	39
6.12.1.4 constFunc() [1/4]	39
6.12.1.5 constFunc() [2/4]	39
6.12.1.6 constFunc() [3/4]	39
6.12.1.7 constFunc() [4/4]	39
6.12.1.8 factorial() [1/2]	40
6.12.1.9 factorial() [2/2]	40
6.12.1.10 FlatIndic() [1/4]	40
6.12.1.11 FlatIndic() [2/4]	40

6.12.1.12 FlatIndic() [3/4]	40
6.12.1.13 FlatIndic() [4/4]	40
6.12.1.14 MPIGlobalToLocal()	41
6.12.1.15 MPILocalToGlobal()	41
6.12.1.16 polyEval()	41
6.12.1.17 polynDerMulti()	41
6.13 /home/renpo/Research/MantleSolver/src/mlweno/util.h File Reference	41
6.13.1 Typedef Documentation	42
6.13.1.1 indice	42
6.13.1.2 indiceSet	42
6.13.1.3 vertex	43
6.13.1.4 vertexSet	43
6.13.2 Function Documentation	43
6.13.2.1 AssignValuesMeshInfo()	43
6.13.2.2 Bend() [1/2]	43
6.13.2.3 Bend() [2/2]	43
6.13.2.4 constFunc() [1/4]	43
6.13.2.5 constFunc() [2/4]	44
6.13.2.6 constFunc() [3/4]	44
6.13.2.7 constFunc() [4/4]	44
6.13.2.8 delete_pointed_to()	44
6.13.2.9 factorial() [1/2]	44
6.13.2.10 factorial() [2/2]	44
6.13.2.11 FlatIndic() [1/4]	45
6.13.2.12 FlatIndic() [2/4]	45
6.13.2.13 FlatIndic() [3/4]	45
6.13.2.14 FlatIndic() [4/4]	45
6.13.2.15 MPIGlobalToLocal()	45
6.13.2.16 MPILocalToGlobal()	45
6.13.2.17 polyEval()	46
6.13.2.18 polynDerMulti()	46
6.14 util.h	46
Index	49

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

MLWENO	Weno reconstruction with a given stencil level	7
------------------------	--	-------------------

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

MLWENO::basisPolynomial	9
MeshInfo	12
MLWENO::multiLevelReconstruction	15
MLWENO::singleLevelReconstruction	19
MLWENO::stencil< T >	22
MLWENO::stencilPolynomial	27

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

/home/renpo/Research/MantleSolver/src/mlweno/mlreconstruction.cpp	31
/home/renpo/Research/MantleSolver/src/mlweno/mlreconstruction.h	31
/home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp	32
/home/renpo/Research/MantleSolver/src/mlweno/polynomial.h	33
/home/renpo/Research/MantleSolver/src/mlweno/slreconstruction.cpp	35
/home/renpo/Research/MantleSolver/src/mlweno/slreconstruction.h	35
/home/renpo/Research/MantleSolver/src/mlweno/stencil.h	36
/home/renpo/Research/MantleSolver/src/mlweno/util.cpp	38
/home/renpo/Research/MantleSolver/src/mlweno/util.h	41

Chapter 4

Namespace Documentation

4.1 MLWENO Namespace Reference

Weno reconstruction with a given stencil level.

Classes

- class [basisPolynomial](#)
- class [multiLevelReconstruction](#)
- class [singleLevelReconstruction](#)
- class [stencil](#)
- class [stencilPolynomial](#)

Enumerations

- enum [stencilType](#) { [cellCentered](#) , [edgeCentered](#) , [vertexCentered](#) }

4.1.1 Detailed Description

Weno reconstruction with a given stencil level.

4.1.2 Enumeration Type Documentation

4.1.2.1 stencilType

```
enum MLWENO::stencilType
```

Enumerator

cellCentered	
edgeCentered	
vertexCentered	

Chapter 5

Class Documentation

5.1 MLWENO::basisPolynomial Class Reference

```
#include <polynomial.h>
```

Public Member Functions

- [basisPolynomial](#) ()
- [basisPolynomial](#) (const int maxDegree[2])
- [basisPolynomial](#) (const int maxDegree[2], double *coef)
- [~basisPolynomial](#) ()
- void [setMaxDegree](#) (const int maxDegree[2])
- void [setCoef](#) (double *coef)
- double * [getCoef](#) () const
- int [getSize](#) () const
- int [getI](#) () const
- int [getJ](#) () const
- double [eval](#) (double x, double y) const
- double [eval](#) ([vertex](#) P) const
- double [operator\(\)](#) (double x, double y) const
- double [operator\(\)](#) ([vertex](#) P) const
- double [der](#) (int derX, int derY, double x, double y) const
- double [der](#) (int derX, int derY, [vertex](#) P) const
- void [printCoef](#) () const
- double [getCoef](#) (int i) const

5.1.1 Constructor & Destructor Documentation

5.1.1.1 basisPolynomial() [1/3]

```
MLWENO::basisPolynomial::basisPolynomial ( ) [inline]
```

5.1.1.2 basisPolynomial() [2/3]

```
basisPolynomial::basisPolynomial (
    const int maxDegree[2] )
```

5.1.1.3 basisPolynomial() [3/3]

```
basisPolynomial::basisPolynomial (
    const int maxDegree[2],
    double * coef )
```

5.1.1.4 ~basisPolynomial()

```
basisPolynomial::~~basisPolynomial ( )
```

5.1.2 Member Function Documentation

5.1.2.1 der() [1/2]

```
double MLWENO::basisPolynomial::der (
    int derX,
    int derY,
    double x,
    double y ) const
```

5.1.2.2 der() [2/2]

```
double MLWENO::basisPolynomial::der (
    int derX,
    int derY,
    vertex P ) const [inline]
```

5.1.2.3 eval() [1/2]

```
double basisPolynomial::eval (
    double x,
    double y ) const
```

5.1.2.4 eval() [2/2]

```
double MLWENO::basisPolynomial::eval (
    vertex P ) const [inline]
```

5.1.2.5 getCoef() [1/2]

```
double * basisPolynomial::getCoef ( ) const
```

5.1.2.6 getCoef() [2/2]

```
double basisPolynomial::getCoef (
    int i ) const
```

5.1.2.7 getI()

```
int MLWENO::basisPolynomial::getI ( ) const [inline]
```

5.1.2.8 getJ()

```
int MLWENO::basisPolynomial::getJ ( ) const [inline]
```

5.1.2.9 getSize()

```
int MLWENO::basisPolynomial::getSize ( ) const [inline]
```

5.1.2.10 operator()() [1/2]

```
double MLWENO::basisPolynomial::operator() (
    double x,
    double y ) const [inline]
```

5.1.2.11 operator() [2/2]

```
double MLWENO::basisPolynomial::operator() (
    vertex P ) const [inline]
```

5.1.2.12 printCoef()

```
void basisPolynomial::printCoef ( ) const
```

5.1.2.13 setCoef()

```
void basisPolynomial::setCoef (
    double * coef )
```

5.1.2.14 setMaxDegree()

```
void basisPolynomial::setMaxDegree (
    const int maxDegree[2] )
```

The documentation for this class was generated from the following files:

- [/home/renpo/Research/MantleSolver/src/mlweno/polynomial.h](#)
- [/home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp](#)

5.2 MeshInfo Struct Reference

```
#include <util.h>
```

Public Attributes

- int [dim](#)
- vector< int > [MPIlocalCellSize](#)
- vector< int > [MPIlocalVertexSize](#)
- [indice MPIlocalCellStart](#)
- [indice MPIlocalVertexStart](#)
- vector< int > [MPIglobalCellSize](#)
- vector< int > [MPIglobalVertexSize](#)
- int [cellGhostLayerSize](#)
- int [vertexGhostLayerSize](#)
- vector< int > [MPIlocalCellSizeFull](#)
- vector< int > [MPIlocalVertexSizeFull](#)
- int [MPIglobalHoriEdgeSize](#)
- int [MPIlocalHoriEdgeSize](#)
- int [MPIglobalVertEdgeSize](#)
- int [MPIlocalVertEdgeSize](#)
- [vertexSet lmesh](#)
- double ** [localVals](#)
- const [indiceSet edgeCorner](#) {{0}, {1}}
- const [indiceSet faceCorner](#) {{0,0},{1,0},{1,1},{0,1}}
- const [indiceSet volumeCorner](#)

5.2.1 Member Data Documentation

5.2.1.1 cellGhostLayerSize

```
int MeshInfo::cellGhostLayerSize
```

5.2.1.2 dim

```
int MeshInfo::dim
```

5.2.1.3 edgeCorner

```
const indiceSet MeshInfo::edgeCorner {{0}, {1}}
```

5.2.1.4 faceCorner

```
const indiceSet MeshInfo::faceCorner {{0,0},{1,0},{1,1},{0,1}}
```

5.2.1.5 lmesh

```
vertexSet MeshInfo::lmesh
```

5.2.1.6 localVals

```
double** MeshInfo::localVals
```

5.2.1.7 MPIglobalCellSize

```
vector<int> MeshInfo::MPIglobalCellSize
```

5.2.1.8 MPIglobalHoriEdgeSize

```
int MeshInfo::MPIglobalHoriEdgeSize
```

5.2.1.9 MPIglobalVertEdgeSize

```
int MeshInfo::MPIglobalVertEdgeSize
```

5.2.1.10 MPIglobalVertexSize

```
vector<int> MeshInfo::MPIglobalVertexSize
```

5.2.1.11 MPIlocalCellSize

```
vector<int> MeshInfo::MPIlocalCellSize
```

5.2.1.12 MPIlocalCellSizeFull

```
vector<int> MeshInfo::MPIlocalCellSizeFull
```

5.2.1.13 MPIlocalCellStart

```
indice MeshInfo::MPIlocalCellStart
```

5.2.1.14 MPIlocalHoriEdgeSize

```
int MeshInfo::MPIlocalHoriEdgeSize
```

5.2.1.15 MPIlocalVertEdgeSize

```
int MeshInfo::MPIlocalVertEdgeSize
```

5.2.1.16 MPIlocalVertexSize

```
vector<int> MeshInfo::MPIlocalVertexSize
```

5.2.1.17 MPIlocalVertexSizeFull

```
vector<int> MeshInfo::MPIlocalVertexSizeFull
```

5.2.1.18 MPIlocalVertexStart

```
indice MeshInfo::MPIlocalVertexStart
```

5.2.1.19 vertexGhostLayerSize

```
int MeshInfo::vertexGhostLayerSize
```

5.2.1.20 volumeCorner

```
const indiceSet MeshInfo::volumeCorner
```

Initial value:

```
{{0,0,0},{1,0,0},{1,1,0},{0,1,0},
                                {0,0,1},{1,0,1},{1,1,1},{0,1,1}}
```

The documentation for this struct was generated from the following file:

- [/home/renpo/Research/MantleSolver/src/mlweno/util.h](#)

5.3 MLWENO::multiLevelReconstruction Class Reference

```
#include <mlreconstruction.h>
```

Public Member Functions

- [multiLevelReconstruction](#) ()
A constructor.
- [multiLevelReconstruction](#) (const [MeshInfo](#) &mi, int stencilSizeX, int stencilSizeY)
- [multiLevelReconstruction](#) (const [MeshInfo](#) &mi, int *stencilSize)
- [multiLevelReconstruction](#) (const [MeshInfo](#) &mi, vector< int * > stencilSizes)
- [~multiLevelReconstruction](#) ()
- void [AddLevel](#) (const [MeshInfo](#) &mi, int stencilSizeX, int stencilSizeY)
A destructor.
- void [AddLevel](#) (const [MeshInfo](#) &mi, int *stencilSize)
- void [AddLevel](#) (const [MeshInfo](#) &mi, vector< int * > stencilSizes)
- void [AddReconstMethod](#) (vector< [indice](#) > brm)
Uniformly add reconstruction methods.
- void [AddReconstMethod](#) (vector< vector< [indice](#) > > brms)
- void [UpdateNonLinearWgts](#) (const [MeshInfo](#) &mi)
Two ways of updating non linear weights with updated smoothness indicators.
- void [UpdateTwoStageNonLinearWgts](#) (const [MeshInfo](#) &mi)
- void [GetInfo](#) ()
Routines used to check results and verification.
- void [PrintSmoothnessIndicator](#) (const [MeshInfo](#) &mi)
- void [PrintNonLinearWgts](#) (const [MeshInfo](#) &mi)
- void [Clear](#) ()

5.3.1 Constructor & Destructor Documentation

5.3.1.1 multiLevelReconstruction() [1/4]

```
MLWENO::multiLevelReconstruction::multiLevelReconstruction ( ) [inline]
```

A constructor.

! Construt multi-level weno reconstruction by specifying each single level

5.3.1.2 multiLevelReconstruction() [2/4]

```
MLWENO::multiLevelReconstruction::multiLevelReconstruction (
    const MeshInfo & mi,
    int stencilSizeX,
    int stencilSizeY ) [inline]
```

5.3.1.3 multiLevelReconstruction() [3/4]

```
MLWENO::multiLevelReconstruction::multiLevelReconstruction (
    const MeshInfo & mi,
    int * stencilSize ) [inline]
```

5.3.1.4 multiLevelReconstruction() [4/4]

```
MLWENO::multiLevelReconstruction::multiLevelReconstruction (
    const MeshInfo & mi,
    vector< int * > stencilSizes ) [inline]
```

5.3.1.5 ~multiLevelReconstruction()

```
MLWENO::multiLevelReconstruction::~~multiLevelReconstruction ( ) [inline]
```

5.3.2 Member Function Documentation

5.3.2.1 AddLevel() [1/3]

```
void MLWENO::multiLevelReconstruction::AddLevel (
    const MeshInfo & mi,
    int * stencilSize ) [inline]
```

5.3.2.2 AddLevel() [2/3]

```
void multiLevelReconstruction::AddLevel (
    const MeshInfo & mi,
    int stencilSizeX,
    int stencilSizeY )
```

A destructor.

Add a single weno reconstruction level to the computation.

! Construt multi-level weno reconstruction by specifying each single level Uniformly add reconstruction level.

5.3.2.3 AddLevel() [3/3]

```
void MLWENO::multiLevelReconstruction::AddLevel (
    const MeshInfo & mi,
    vector< int * > stencilSizes ) [inline]
```

5.3.2.4 AddReconstMethod() [1/2]

```
void MLWENO::multiLevelReconstruction::AddReconstMethod (
    vector< indice > brm ) [inline]
```

Uniformly add reconstruction methods.

5.3.2.5 AddReconstMethod() [2/2]

```
void MLWENO::multiLevelReconstruction::AddReconstMethod (
    vector< vector< indice > > brms ) [inline]
```

5.3.2.6 Clear()

```
void multiLevelReconstruction::Clear ( )
```

5.3.2.7 GetInfo()

```
void multiLevelReconstruction::GetInfo ( )
```

Routines used to check results and verification.

5.3.2.8 PrintNonLinearWgts()

```
void multiLevelReconstruction::PrintNonLinearWgts (
    const MeshInfo & mi )
```

5.3.2.9 PrintSmoothnessIndicator()

```
void multiLevelReconstruction::PrintSmoothnessIndicator (
    const MeshInfo & mi )
```

5.3.2.10 UpdateNonLinearWgts()

```
void multiLevelReconstruction::UpdateNonLinearWgts (
    const MeshInfo & mi )
```

Two ways of updateing non linear weights with updated smoothness indicators.

5.3.2.11 UpdateTwoStageNonLinearWgts()

```
void multiLevelReconstruction::UpdateTwoStageNonLinearWgts (
    const MeshInfo & mi )
```

The documentation for this class was generated from the following files:

- [/home/renpo/Research/MantleSolver/src/mlweno/mlreconstruction.h](#)
- [/home/renpo/Research/MantleSolver/src/mlweno/mlreconstruction.cpp](#)

5.4 MLWENO::singleLevelReconstruction Class Reference

```
#include <slreconstruction.h>
```

Public Member Functions

- [singleLevelReconstruction \(\)](#)
A constructor.
- [singleLevelReconstruction \(int stencilSizeX, int stencilSizeY\)](#)
A constructor.
- [~singleLevelReconstruction \(\)](#)
A destructor.
- void [CreateStencilPolynomials](#) (const [MeshInfo](#) &mi)
- bool [CheckExist](#) (const [MeshInfo](#) &mi, [indice](#) owner) const
- const int [GetSizeX](#) () const
- const int [GetSizeY](#) () const
- const double [GetScale](#) (int s)
- const double [GetScale](#) (const [MeshInfo](#) &mi, [indice](#) owner)
- double [CalculateSmoothnessIndic](#) (const [MeshInfo](#) &mi, [indice](#) owner)
Calculate smoothness indicator for the single stencil.
- void [CheckStencils](#) () const
class members for checking and verification
- void [CheckStencilPolynomials](#) (const [MeshInfo](#) &mi, [indice](#) start)
Functions checking created stencil polynomials for a single stencil.
- void [PrintSmoothnessIndicator](#) (const [MeshInfo](#) &mi)
Print all smoothnessIndicator for all stencils belonging to this level.

5.4.1 Constructor & Destructor Documentation

5.4.1.1 singleLevelReconstruction() [1/2]

```
MLWENO::singleLevelReconstruction::singleLevelReconstruction ( ) [inline]
```

A constructor.

Default constructor for a single reconstruction level.

5.4.1.2 singleLevelReconstruction() [2/2]

```
singleLevelReconstruction::singleLevelReconstruction (
    int stencilSizeX,
    int stencilSizeY )
```

A constructor.

Costume constructor with given x and y stencil size. Create stencil of indice used in creating stencil polynomials

5.4.1.3 ~singleLevelReconstruction()

```
MLWENO::singleLevelReconstruction::~~singleLevelReconstruction ( ) [inline]
```

A destructor.

Clear interior, single level polynomial and smoothness indicators

5.4.2 Member Function Documentation

5.4.2.1 CalculateSmoothnessIndic()

```
double singleLevelReconstruction::CalculateSmoothnessIndic (
    const MeshInfo & mi,
    indice owner )
```

Calculate smoothness indicator for the single stencil.

Directly calculate smoothness indicator of a given stencil Should not be called directly for computational efficiency

5.4.2.2 CheckExist()

```
bool MLWENO::singleLevelReconstruction::CheckExist (
    const MeshInfo & mi,
    indice owner ) const [inline]
```

5.4.2.3 CheckStencilPolynomials()

```
void singleLevelReconstruction::CheckStencilPolynomials (
    const MeshInfo & mi,
    indice start )
```

Functions checking created stencil polynomials for a single stencil.

5.4.2.4 CheckStencils()

```
void MLWENO::singleLevelReconstruction::CheckStencils ( ) const [inline]
```

class members for checking and verification

5.4.2.5 CreateStencilPolynomials()

```
void singleLevelReconstruction::CreateStencilPolynomials (
    const MeshInfo & mi )
```

! Two steps of creating stencil polynomials for a given reconstruction level. Step 1: Identify all the stencils covering entire computational domain. Step 2: Compute stencil polynomials for each stencil.

5.4.2.6 GetScale() [1/2]

```
const double MLWENO::singleLevelReconstruction::GetScale (
    const MeshInfo & mi,
    indice owner ) [inline]
```

5.4.2.7 GetScale() [2/2]

```
const double MLWENO::singleLevelReconstruction::GetScale (
    int s ) [inline]
```

5.4.2.8 GetSizeX()

```
const int MLWENO::singleLevelReconstruction::GetSizeX ( ) const [inline]
```

5.4.2.9 GetSizeY()

```
const int MLWENO::singleLevelReconstruction::GetSizeY ( ) const [inline]
```

5.4.2.10 PrintSmoothnessIndicator()

```
void singleLevelReconstruction::PrintSmoothnessIndicator (
    const MeshInfo & mi )
```

Print all smoothnessIndicator for all stencils belonging to this level.

The documentation for this class was generated from the following files:

- /home/renpo/Research/MantleSolver/src/mlweno/[slreconstruction.h](#)
- /home/renpo/Research/MantleSolver/src/mlweno/[slreconstruction.cpp](#)

5.5 MLWENO::stencil< T > Class Template Reference

```
#include <stencil.h>
```

Public Member Functions

- [stencil](#) ()
- [stencil](#) (size_t I)
- [stencil](#) (size_t I, size_t J)
- [stencil](#) (size_t I, size_t J, size_t K)
- [~stencil](#) ()
- void [SetStencil](#) (size_t I)
- void [SetStencil](#) (size_t I, size_t J)
- void [SetStencil](#) (size_t I, size_t J, size_t K)
- void [CreateStencil](#) ()
- const int [getSize](#) () const
- const int [getI](#) () const
- const int [getJ](#) () const
- const int [getK](#) () const
- void [setI](#) (size_t I) const
- void [setJ](#) (size_t J) const
- void [setK](#) (size_t K) const
- T & [operator\(\)](#) (size_t i)
- T & [operator\(\)](#) (size_t i, size_t j)
- T & [operator\(\)](#) (size_t i, size_t j, size_t k)
- T [operator\(\)](#) (size_t i) const
- T [operator\(\)](#) (size_t i, size_t j) const
- T [operator\(\)](#) (size_t i, size_t j, size_t k) const
- const vector< T > & [GetStencil](#) () const
- void [clearPtr](#) ()

5.5.1 Constructor & Destructor Documentation

5.5.1.1 stencil() [1/4]

```
template<class T >
MLWENO::stencil< T >::stencil ( ) [inline]
```

5.5.1.2 stencil() [2/4]

```
template<class T >
MLWENO::stencil< T >::stencil (
    size_t I ) [inline]
```

5.5.1.3 stencil() [3/4]

```
template<class T >
MLWENO::stencil< T >::stencil (
    size_t I,
    size_t J ) [inline]
```

5.5.1.4 stencil() [4/4]

```
template<class T >
MLWENO::stencil< T >::stencil (
    size_t I,
    size_t J,
    size_t K ) [inline]
```

5.5.1.5 ~stencil()

```
template<class T >
MLWENO::stencil< T >::~~stencil ( ) [inline]
```

5.5.2 Member Function Documentation

5.5.2.1 clearPtr()

```
template<class T >
void MLWENO::stencil< T >::clearPtr ( ) [inline]
```

5.5.2.2 CreateStencil()

```
template<class T >
void MLWENO::stencil< T >::CreateStencil ( ) [inline]
```

5.5.2.3 getI()

```
template<class T >
const int MLWENO::stencil< T >::getI ( ) const [inline]
```

5.5.2.4 getJ()

```
template<class T >
const int MLWENO::stencil< T >::getJ ( ) const [inline]
```

5.5.2.5 getK()

```
template<class T >
const int MLWENO::stencil< T >::getK ( ) const [inline]
```

5.5.2.6 getSize()

```
template<class T >
const int MLWENO::stencil< T >::getSize ( ) const [inline]
```

5.5.2.7 GetStencil()

```
template<class T >
const vector< T > & MLWENO::stencil< T >::GetStencil ( ) const [inline]
```

5.5.2.8 operator>() [1/6]

```
template<class T >
T & MLWENO::stencil< T >::operator() (
    size_t i ) [inline]
```

5.5.2.9 operator>() [2/6]

```
template<class T >
T MLWENO::stencil< T >::operator() (
    size_t i ) const [inline]
```

5.5.2.10 operator>() [3/6]

```
template<class T >
T & MLWENO::stencil< T >::operator() (
    size_t i,
    size_t j ) [inline]
```

5.5.2.11 operator>() [4/6]

```
template<class T >
T MLWENO::stencil< T >::operator() (
    size_t i,
    size_t j ) const [inline]
```

5.5.2.12 operator>() [5/6]

```
template<class T >
T & MLWENO::stencil< T >::operator() (
    size_t i,
    size_t j,
    size_t k ) [inline]
```

5.5.2.13 operator>() [6/6]

```
template<class T >
T MLWENO::stencil< T >::operator() (
    size_t i,
    size_t j,
    size_t k ) const [inline]
```

5.5.2.14 setI()

```
template<class T >
void MLWENO::stencil< T >::setI (
    size_t I ) const [inline]
```

5.5.2.15 setJ()

```
template<class T >
void MLWENO::stencil< T >::setJ (
    size_t J ) const [inline]
```

5.5.2.16 setK()

```
template<class T >
void MLWENO::stencil< T >::setK (
    size_t K ) const [inline]
```

5.5.2.17 SetStencil() [1/3]

```
template<class T >
void MLWENO::stencil< T >::SetStencil (
    size_t I ) [inline]
```

5.5.2.18 SetStencil() [2/3]

```
template<class T >
void MLWENO::stencil< T >::SetStencil (
    size_t I,
    size_t J ) [inline]
```

5.5.2.19 SetStencil() [3/3]

```
template<class T >
void MLWENO::stencil< T >::SetStencil (
    size_t I,
    size_t J,
    size_t K ) [inline]
```

The documentation for this class was generated from the following file:

- </home/renpo/Research/MantleSolver/src/mlweno/stencil.h>

5.6 MLWENO::stencilPolynomial Class Reference

```
#include <polynomial.h>
```

Public Member Functions

- [stencilPolynomial](#) ()
- [stencilPolynomial](#) (const [indice](#) &start, const [vertex](#) ¢er)
- [stencilPolynomial](#) (const [indice](#) &start, const [vertex](#) ¢er, const vector< [indice](#) > &targetCell)
- [~stencilPolynomial](#) ()
- void [SetStencilPolynomials](#) (const [MeshInfo](#) &mi, const [stencil](#)< [indice](#) > &stencilIndice)
- void [SetUpScale](#) (const [MeshInfo](#) &mi, const vector< [indice](#) > &targetCell)
- void [SetUpScale](#) (const [MeshInfo](#) &mi, const [stencil](#)< [indice](#) > &stencilIndice)
- const double [GetScale](#) () const
- double [eval](#) (const double x, const double y) const
- double [eval](#) (const [vertex](#) &P) const
- double [operator\(\)](#) (const double x, const double y) const
- double [operator\(\)](#) (const [vertex](#) &P) const
- void [printCoef](#) ()
- void [printCoef](#) (int s)
- void [SetCollapsePolyn](#) (const [MeshInfo](#) &mi, const [stencil](#)< [indice](#) > &stencilIndice)
- double [GetSmoothIndic](#) (const [MeshInfo](#) &mi, const [stencil](#)< [indice](#) > &stencilIndice)
- const int [GetOrderX](#) () const
- const int [GetOrderY](#) () const

5.6.1 Constructor & Destructor Documentation

5.6.1.1 stencilPolynomial() [1/3]

```
MLWENO::stencilPolynomial::stencilPolynomial ( ) [inline]
```

5.6.1.2 stencilPolynomial() [2/3]

```
stencilPolynomial::stencilPolynomial (
    const indice & start,
    const vertex & center )
```

5.6.1.3 stencilPolynomial() [3/3]

```
stencilPolynomial::stencilPolynomial (
    const indice & start,
    const vertex & center,
    const vector< indice > & targetCell )
```

5.6.1.4 ~stencilPolynomial()

```
MLWENO::stencilPolynomial::~~stencilPolynomial ( ) [inline]
```

5.6.2 Member Function Documentation

5.6.2.1 eval() [1/2]

```
double stencilPolynomial::eval (
    const double x,
    const double y ) const
```

5.6.2.2 eval() [2/2]

```
double MLWENO::stencilPolynomial::eval (
    const vertex & P ) const [inline]
```

5.6.2.3 GetOrderX()

```
const int MLWENO::stencilPolynomial::GetOrderX ( ) const [inline]
```

5.6.2.4 GetOrderY()

```
const int MLWENO::stencilPolynomial::GetOrderY ( ) const [inline]
```

5.6.2.5 GetScale()

```
const double MLWENO::stencilPolynomial::GetScale ( ) const [inline]
```

5.6.2.6 GetSmoothIndic()

```
double stencilPolynomial::GetSmoothIndic (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndices )
```

5.6.2.7 operator() [1/2]

```
double MLWENO::stencilPolynomial::operator() (
    const double x,
    const double y ) const [inline]
```

5.6.2.8 operator() [2/2]

```
double MLWENO::stencilPolynomial::operator() (
    const vertex & P ) const [inline]
```

5.6.2.9 printCoef() [1/2]

```
void stencilPolynomial::printCoef ( )
```

5.6.2.10 printCoef() [2/2]

```
void stencilPolynomial::printCoef (
    int s )
```

5.6.2.11 SetCollapsePolyn()

```
void stencilPolynomial::SetCollapsePolyn (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndice )
```

5.6.2.12 SetStencilPolynomials()

```
void stencilPolynomial::SetStencilPolynomials (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndice )
```

5.6.2.13 `SetUpScale()` [1/2]

```
void MLWENO::stencilPolynomial::SetUpScale (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndices ) [inline]
```

5.6.2.14 `SetUpScale()` [2/2]

```
void MLWENO::stencilPolynomial::SetUpScale (
    const MeshInfo & mi,
    const vector< indice > & targetCell ) [inline]
```

The documentation for this class was generated from the following files:

- [/home/renpo/Research/MantleSolver/src/mlweno/polynomial.h](#)
- [/home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp](#)

Chapter 6

File Documentation

6.1 /home/renpo/Research/Mantle↵ Solver/src/mlweno/mlreconstruction.cpp File Reference

```
#include "mlreconstruction.h"
```

6.2 /home/renpo/Research/MantleSolver/src/mlweno/mlreconstruction.h File Reference

```
#include "slreconstruction.h"
```

Classes

- class [MLWENO::multiLevelReconstruction](#)

Namespaces

- namespace [MLWENO](#)
Weno reconstruction with a given stencil level.

6.3 mlreconstruction.h

[Go to the documentation of this file.](#)

```

00001 #ifndef MLRECONSTRUCTION_H_
00002 #define MLRECONSTRUCTION_H_
00003
00004 #include "slreconstruction.h"
00005
00006 namespace MLWENO{
00007     class multiLevelReconstruction {
00008     public:
00009
00010         multiLevelReconstruction() {};
00011         multiLevelReconstruction(const MeshInfo& mi, int stencilSizeX, int
00012 stencilSizeY){AddLevel(mi,stencilSizeX,stencilSizeY);};
00013         multiLevelReconstruction(const MeshInfo& mi, int* stencilSize)
00014 {AddLevel(mi,stencilSize);};
00015         multiLevelReconstruction(const MeshInfo& mi, vector<int*> stencilSizes)
00016 {AddLevel(mi,stencilSizes);};
00017         ~multiLevelReconstruction() {Clear();};
00018
00019         void AddLevel(const MeshInfo& mi, int stencilSizeX, int stencilSizeY);
00020         void AddLevel(const MeshInfo& mi, int* stencilSize)
00021 {AddLevel(mi,stencilSize[0],stencilSize[1]);};
00022         void AddLevel(const MeshInfo& mi, vector<int*> stencilSizes)
00023 {for (int i=0; i<stencilSizes.size(); i++){
00024     AddLevel(mi,stencilSizes[i]);};};
00025         void AddReconstMethod(vector<indice> brm) {baseReconstMethod_.push_back(brm);
00026 AddWgts_();};
00027         void AddReconstMethod(vector<vector<indice> brms)
00028 {for (int i=0; i<brms.size(); i++){
00029     AddReconstMethod(brms[i]);
00030 };};
00031
00032         void UpdateNonLinearWgts(const MeshInfo& mi);
00033         void UpdateTwoStageNonLinearWgts(const MeshInfo& mi);
00034
00035         void GetInfo();
00036         void PrintSmoothnessIndicator(const MeshInfo& mi);
00037         void PrintNonLinearWgts(const MeshInfo& mi);
00038
00039         void Clear();
00040     private:
00041
00042         const double eps0_ = 0.01;
00043
00044         vector< singleLevelReconstruction *> allLevels_;
00045         vector<vector<indice> baseReconstMethod_;
00046         vector< map<int, double> > linearWgts_;
00047
00048         map <int, vector<vector<indice>*> > reconstMethods_;
00049         map <int, vector< map<int, double>*> > allLinearWgts_;
00050         map<int, vector< map<int, double> > > nonLinearWgts_;
00051
00052         vector< map<int, int> > etaBias_;
00053
00054         // Collective methods
00055         void ResetWgts_();
00056         void AddWgts_();
00057
00058         void UpdateNonLinearWgts_(const MeshInfo& mi, indice start);
00059         void UpdateFirstStageNonLinearWgts_(const MeshInfo& mi, indice start);
00060         void UpdateTwoStageNonLinearWgts_(const MeshInfo& mi, indice start);
00061
00062     };
00063 }
00064 #endif

```

6.4 /home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp File Reference

```
#include "polynomial.h"
```

Functions

- double [basePoly](#) ([vertex](#) &point, const vector< int > ¶m)

6.4.1 Function Documentation

6.4.1.1 basePoly()

```
double basePoly (
    vertex & point,
    const vector< int > & param )
```

6.5 /home/renpo/Research/MantleSolver/src/mlweno/polynomial.h File Reference

```
#include "util.h"
#include "stencil.h"
```

Classes

- class [MLWENO::basisPolynomial](#)
- class [MLWENO::stencilPolynomial](#)

Namespaces

- namespace [MLWENO](#)
Weno reconstruction with a given stencil level.

6.6 polynomial.h

[Go to the documentation of this file.](#)

```
00001 #ifndef POLYNOMIAL_H_
00002 #define POLYNOMIAL_H_
00003
00004 #include "util.h"
00005 #include "stencil.h"
00006
00007 namespace MLWENO{
00008     // Tensor product 2D polynomial for stencil basis
00009     class basisPolynomial {
00010     public:
00011         basisPolynomial() {};
00012         basisPolynomial(const int maxDegree[2]);
00013         basisPolynomial(const int maxDegree[2], double* coef);
00014         ~basisPolynomial();
00015
00016         void setMaxDegree(const int maxDegree[2]);
00017         void setCoef(double* coef);
```

```

00019         double* getCoef() const;
00020
00021
00022         // Return the size of the polynomial
00023         int getSize() const {return maxDegree_[0]*maxDegree_[1];};
00024
00025         // Return the size of degree x as I
00026         int getI() const {return maxDegree_[0];};
00027         // Return the size of degree y as J
00028         int getJ() const {return maxDegree_[1];};
00029
00030         // Evaluation of point value for a given polynomial
00031         double eval(double x, double y) const;
00032         double eval(vertex P) const {return eval(P[0],P[1]);};
00033         double operator() (double x, double y) const {return eval(x,y);};
00034         double operator() (vertex P) const {return eval(P);};
00035
00036         // Evaluation of derivative value for a given polynomial
00037         double der(int derX, int derY, double x, double y) const;
00038         double der(int derX, int derY, vertex P) const {return der(derX, derY, P[0], P[1]);};
00039
00040         // Print coefficients out
00041         void printCoef() const;
00042
00043         double getCoef(int i) const;
00044
00045     private:
00046
00047         int maxDegree_[2] = {-1,-1};
00048         double* coef_ = nullptr;
00049 };
00050
00051 // stencil polynomials used for weno reconstruction
00052 class stencilPolynomial {
00053     public:
00054         stencilPolynomial() {};
00055         stencilPolynomial(const indice& start, const vertex& center);
00056         stencilPolynomial(const indice& start, const vertex& center,
00057                         const vector<indice>& targetCell);
00058         ~stencilPolynomial() {stencilPolyn_.clearPtr();
00059                             delete [] collapsePolyn_};
00060
00061         void SetStencilPolynomials(const MeshInfo& mi,
00062                                   const stencil <indice>& stencilIndice);
00063
00064         void SetUpScale(const MeshInfo& mi, const vector<indice>& targetCell)
00065             {SetTargetCell_(targetCell);ComputeCellBasedScale_(mi);};
00066         void SetUpScale(const MeshInfo& mi, const stencil <indice>& stencilIndice)
00067             {ComputeStencilBasedScale_(mi, stencilIndice);};
00068
00069         const double GetScale() const {return scale_};
00070
00071         double eval(const double x, const double y) const;
00072         double eval(const vertex& P) const {return eval(P[0],P[1]);};
00073         double operator() (const double x, const double y) const {return eval(x,y);};
00074         double operator() (const vertex& P) const {return eval(P);};
00075
00076         // Check basis polynomial coefficients
00077         void printCoef();
00078         void printCoef(int s);
00079
00080         // Set collapse polynomial
00081         void SetCollapsePolyn(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00082
00083         double GetSmoothIndic(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00084
00085         const int GetOrderX() const { return stencilPolyn_.getI();};
00086         const int GetOrderY() const { return stencilPolyn_.getJ();};
00087
00088     private:
00089
00090         vertex center_ = {0.0,0.0};
00091         indice start_ = {-1,-1};
00092
00093         void SetTargetCell_(const vector<indice>& targetCell);
00094         vector<indice> targetCell_;
00095         double scale_ = -1.0;
00096
00097         double coef_ = 1.0;
00098
00099         // Calculate scale based on the information of a single cell
00100         void ComputeCellBasedScale_(const MeshInfo& mi);
00101         // Calculate scale based on the value of a given stencil
00102         void ComputeStencilBasedScale_(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00103
00104         stencil <basisPolynomial*> stencilPolyn_;

```

```

00105
00106         void SetCollapsePolyn_(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00107         basisPolynomial* collapsePolyn_ = nullptr;
00108
00109         // Create polynomial smoothness indicator
00110         void CreateXi_();
00111         vector<double> Xi_;
00112
00113         int maxR_;
00114         int minR_;
00115
00116         void EvalSmoothIndic_(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00117         double smoothnessIndic_ = -1;
00118     };
00119
00120 // End of using name space MLWENO
00121 }
00122
00123 #endif

```

6.7 /home/renpo/Research/MantleSolver/src/mlweno/slreconstruction.cpp File Reference

```
#include "slreconstruction.h"
```

6.8 /home/renpo/Research/MantleSolver/src/mlweno/slreconstruction.h File Reference

```
#include "polynomial.h"
#include <map>
```

Classes

- class [MLWENO::singleLevelReconstruction](#)

Namespaces

- namespace [MLWENO](#)
Weno reconstruction with a given stencil level.

6.9 slreconstruction.h

[Go to the documentation of this file.](#)

```

00001 #ifndef SLRECONSTRUCTION_H_
00002 #define SLRECONSTRUCTION_H_
00003
00004 //\slreconstruction.h slreconstruction.h
00005
00006
00007 #include "polynomial.h"
00008 #include <map>
00009
00010 namespace MLWENO{
00011
00012     class singleLevelReconstruction {

```

```

00013         public:
00015
00018             singleLevelReconstruction() {};
00020
00023             singleLevelReconstruction(int stencilSizeX, int stencilSizeY);
00024
00026
00029             ~singleLevelReconstruction() {interior_.clear();
singleLevel_.clear();smoothnessIndic_.clear();};
00030
00031             // =====
00032             void CreateStencilPolynomials(const MeshInfo& mi);
00033
00034             bool CheckExist(const MeshInfo& mi, indice owner) const {return
interior_.count(FlatIndic(mi,owner));};
00035
00036             const int GetSizeX() const {return stencilSizeX_};
00037             const int GetSizeY() const {return stencilSizeY_};
00038
00039             const double GetScale(int s) {return singleLevel_[s]->GetScale();};
00040             const double GetScale(const MeshInfo& mi, indice owner) {return
singleLevel_[FlatIndic(mi,owner)]->GetScale();};
00041
00044             double CalculateSmoothnessIndic(const MeshInfo& mi, indice owner);
00045
00046             // =====
00048             void CheckStencils() const {cout<< "Constructed "<< interior_.size() << " stencils with the
size of " << stencilSizeX_ << " " << stencilSizeY_ << endl;};
00049             void CheckStencilPolynomials(const MeshInfo& mi, indice start);
00050             void PrintSmoothnessIndicator(const MeshInfo& mi);
00051
00052         private:
00053
00054             int stencilSizeX_ = -1;
00055             int stencilSizeY_ = -1;
00056
00057             stencil<indice> stencilIndice_;
00058             unordered_set<int> interior_;
00059             map<int, stencilPolynomial*> singleLevel_;
00060             map<int, double> smoothnessIndic_;
00061
00062             void IdentifyInteriorCell_(const MeshInfo& mi);
00063
00064             const vertex ComputeStencilCenter_(const MeshInfo& mi, int flat);
00065
00066             void ComputeStencilPolyn_(const MeshInfo& mi);
00067
00068             void UpdateSmoothnessIndic_(const MeshInfo& mi);
00069
00070     };
00071
00072 }
00073
00074 #endif

```

6.10 /home/renpo/Research/MantleSolver/src/mlweno/stencil.h File Reference

```
#include "util.h"
```

Classes

- class [MLWENO::stencil< T >](#)

Namespaces

- namespace [MLWENO](#)

Weno reconstruction with a given stencil level.

Enumerations

- enum `MLWENO::stencilType` { `MLWENO::cellCentered`, `MLWENO::edgeCentered`, `MLWENO::vertexCentered` }

6.11 stencil.h

Go to the documentation of this file.

```

00001 #ifndef STENCIL_H_
00002 #define STENCIL_H_
00003
00004 /*
00005  *Define multi-level stencils for weno reconstruction
00006  *Designed for cartesian logically rectangular mesh.
00007  */
00008
00009 #include "util.h"
00010
00011 namespace MLWENO {
00012
00013     enum stencilType {cellCentered, edgeCentered, vertexCentered};
00014
00015     template <class T>
00016     class stencil{
00017     public:
00018         stencil() {};
00019         stencil(size_t I): I_(I), stencilSize_(I)
00020 {stencil_.resize(stencilSize_);};
00021         stencil(size_t I, size_t J): I_(I), J_(J), stencilSize_(I*J)
00022 {stencil_.resize(stencilSize_);};
00023         stencil(size_t I, size_t J, size_t K): I_(I), J_(J), K_(K), stencilSize_(I*J*K)
00024 {stencil_.resize(stencilSize_);};
00025
00026         ~stencil() {};
00027
00028         void SetStencil(size_t I) {I_ = I; stencilSize_ = I; stencil_.resize(stencilSize_);};
00029         void SetStencil(size_t I, size_t J) {I_ = I; J_ = J; stencilSize_ = I*J;
00030 stencil_.resize(stencilSize_);};
00031         void SetStencil(size_t I, size_t J, size_t K) {I_ = I; J_ = J; K_ = K; stencilSize_ =
00032 I*J*K; stencil_.resize(stencilSize_);};
00033
00034         void CreateStencil() {stencil_.resize(stencilSize_);};
00035
00036         const int getSize() const {return stencilSize_};
00037
00038         const int getI() const {return I_};
00039         const int getJ() const {return J_};
00040         const int getK() const {return K_};
00041
00042         void setI(size_t I) const {I_ = I;};
00043         void setJ(size_t J) const {J_ = J;};
00044         void setK(size_t K) const {K_ = K;};
00045
00046         T &operator()(size_t i){
00047             return stencil_[i];
00048         }
00049
00050         T &operator()(size_t i, size_t j){
00051             return stencil_[i+j*I_];
00052         }
00053
00054         T &operator()(size_t i, size_t j, size_t k){
00055             return stencil_[i+j*I_+k*I_*J_];
00056         }
00057
00058         T operator()(size_t i) const{
00059             return stencil_[i];
00060         }
00061
00062         T operator()(size_t i, size_t j) const{
00063             return stencil_[i+j*I_];
00064         }
00065
00066         T operator()(size_t i, size_t j, size_t k) const{
00067             return stencil_[i+j*I_+k*I_*J_];
00068         }
00069
00070         const vector<T>& GetStencil() const {return stencil_;}
00071
00072         // Call it when T is pointer

```

```

00068         void clearPtr() {if (std::is_pointer<T>::value){
00069             std::for_each(stencil_.begin(),stencil_.end(),delete_pointed_to<T>);}}
00070
00071         private:
00072             size_t I_;
00073             size_t J_;
00074             size_t K_;
00075             size_t stencilSize_;
00076             vector<T> stencil_;
00077         };
00078
00079 // End of using name space MLWENO
00080 }
00081
00082 #endif

```

6.12 /home/renpo/Research/MantleSolver/src/mlweno/util.cpp File Reference

```
#include "util.h"
```

Functions

- double [constFunc](#) (valarray< double > &point, const vector< double > ¶m)
- double [constFunc](#) (valarray< double > &point, const vector< double > ¶m, double c)
- double [constFunc](#) ()
- double [constFunc](#) (double c)
- int [factorial](#) (int top, int bottom)
- int [factorial](#) (int top)
- double [polyEval](#) (double x, double *coef, int degree)
- void [polynDerMulti](#) (int der, int max, int *multiplier)
- [indice MPILocalToGlobal](#) ([indice](#) local, const [MeshInfo](#) &mi)
- [indice MPIGlobalToLocal](#) ([indice](#) global, const [MeshInfo](#) &mi)
- int [FlatIndic](#) (const [MeshInfo](#) &mi, int i, int j)
- int [FlatIndic](#) (const int M, int i, int j)
- int [FlatIndic](#) (const [MeshInfo](#) &mi, const [indice](#) &p)
- int [FlatIndic](#) (const int M, const [indice](#) &p)
- [indice Bend](#) (const [MeshInfo](#) &mi, int flat)
- [indice Bend](#) (const int M, int flat)
- void [AssignValuesMeshInfo](#) ([MeshInfo](#) &mi, DM dmv, DM dmu)

6.12.1 Function Documentation

6.12.1.1 AssignValuesMeshInfo()

```

void AssignValuesMeshInfo (
    MeshInfo & mi,
    DM dmv,
    DM dmu )

```

6.12.1.2 Bend() [1/2]

```
indice Bend (
    const int M,
    int flat )
```

6.12.1.3 Bend() [2/2]

```
indice Bend (
    const MeshInfo & mi,
    int flat )
```

6.12.1.4 constFunc() [1/4]

```
double constFunc ( )
```

6.12.1.5 constFunc() [2/4]

```
double constFunc (
    double c )
```

6.12.1.6 constFunc() [3/4]

```
double constFunc (
    valarray< double > & point,
    const vector< double > & param )
```

6.12.1.7 constFunc() [4/4]

```
double constFunc (
    valarray< double > & point,
    const vector< double > & param,
    double c )
```

6.12.1.8 factorial() [1/2]

```
int factorial (
    int top )
```

6.12.1.9 factorial() [2/2]

```
int factorial (
    int top,
    int bottom )
```

6.12.1.10 FlatIndic() [1/4]

```
int FlatIndic (
    const int M,
    const indice & p )
```

6.12.1.11 FlatIndic() [2/4]

```
int FlatIndic (
    const int M,
    int i,
    int j )
```

6.12.1.12 FlatIndic() [3/4]

```
int FlatIndic (
    const MeshInfo & mi,
    const indice & p )
```

6.12.1.13 FlatIndic() [4/4]

```
int FlatIndic (
    const MeshInfo & mi,
    int i,
    int j )
```

6.12.1.14 MPIGlobalToLocal()

```
indice MPIGlobalToLocal (
    indice global,
    const MeshInfo & mi )
```

6.12.1.15 MPILocalToGlobal()

```
indice MPILocalToGlobal (
    indice local,
    const MeshInfo & mi )
```

6.12.1.16 polyEval()

```
double polyEval (
    double x,
    double * coef,
    int degree )
```

6.12.1.17 polynDerMulti()

```
void polynDerMulti (
    int der,
    int max,
    int * multiplier )
```

6.13 /home/renpo/Research/MantleSolver/src/mlweno/util.h File Reference

```
#include <vector>
#include <utility>
#include <set>
#include <unordered_set>
#include <array>
#include <valarray>
#include <algorithm>
#include <numeric>
#include <memory>
#include <cstdlib>
#include <type_traits>
#include <iomanip>
#include "lapacke.h"
#include "integral.h"
#include <assert.h>
#include <petsc.h>
```

Classes

- struct [MeshInfo](#)

Typedefs

- using [vertex](#) = valarray< double >
- using [indice](#) = valarray< int >
- using [vertexSet](#) = vector< valarray< double > >
- using [indiceSet](#) = vector< valarray< double > >

Functions

- template<typename T >
void [delete_pointed_to](#) (T const ptr)
- void [AssignValuesMeshInfo](#) ([MeshInfo](#) &mi, DM dmv, DM dmu)
- double [constFunc](#) (valarray< double > &point, const vector< double > ¶m)
- double [constFunc](#) (valarray< double > &point, const vector< double > ¶m, double c)
- double [constFunc](#) ()
- double [constFunc](#) (double c)
- int [factorial](#) (int top, int bottom)
- int [factorial](#) (int top)
- double [polyEval](#) (double x, double *coef, int degree)
- void [polynDerMulti](#) (int der, int max, int *multiplier)
- [indice MPILocalToGlobal](#) ([indice](#) local, const [MeshInfo](#) &mi)
- [indice MPIGlobalToLocal](#) ([indice](#) global, const [MeshInfo](#) &mi)
- int [FlatIndic](#) (const [MeshInfo](#) &mi, int i, int j)
- int [FlatIndic](#) (const int M, int i, int j)
- int [FlatIndic](#) (const [MeshInfo](#) &mi, const [indice](#) &p)
- int [FlatIndic](#) (const int M, const [indice](#) &p)
- [indice Bend](#) (const [MeshInfo](#) &mi, int flat)
- [indice Bend](#) (const int M, int flat)

6.13.1 Typedef Documentation

6.13.1.1 indice

```
using indice = valarray<int>
```

6.13.1.2 indiceSet

```
using indiceSet = vector<valarray<double> >
```

6.13.1.3 vertex

```
using vertex = valarray<double>
```

6.13.1.4 vertexSet

```
using vertexSet = vector<valarray<double> >
```

6.13.2 Function Documentation

6.13.2.1 AssignValuesMeshInfo()

```
void AssignValuesMeshInfo (
    MeshInfo & mi,
    DM dmv,
    DM dmu )
```

6.13.2.2 Bend() [1/2]

```
indice Bend (
    const int M,
    int flat )
```

6.13.2.3 Bend() [2/2]

```
indice Bend (
    const MeshInfo & mi,
    int flat )
```

6.13.2.4 constFunc() [1/4]

```
double constFunc ( )
```

6.13.2.5 constFunc() [2/4]

```
double constFunc (
    double c )
```

6.13.2.6 constFunc() [3/4]

```
double constFunc (
    valarray< double > & point,
    const vector< double > & param )
```

6.13.2.7 constFunc() [4/4]

```
double constFunc (
    valarray< double > & point,
    const vector< double > & param,
    double c )
```

6.13.2.8 delete_pointed_to()

```
template<typename T >
void delete_pointed_to (
    T const ptr )
```

6.13.2.9 factorial() [1/2]

```
int factorial (
    int top )
```

6.13.2.10 factorial() [2/2]

```
int factorial (
    int top,
    int bottom )
```

6.13.2.11 FlatIndic() [1/4]

```
int FlatIndic (
    const int M,
    const indice & p )
```

6.13.2.12 FlatIndic() [2/4]

```
int FlatIndic (
    const int M,
    int i,
    int j )
```

6.13.2.13 FlatIndic() [3/4]

```
int FlatIndic (
    const MeshInfo & mi,
    const indice & p )
```

6.13.2.14 FlatIndic() [4/4]

```
int FlatIndic (
    const MeshInfo & mi,
    int i,
    int j )
```

6.13.2.15 MPIGlobalToLocal()

```
indice MPIGlobalToLocal (
    indice global,
    const MeshInfo & mi )
```

6.13.2.16 MPILocalToGlobal()

```
indice MPILocalToGlobal (
    indice local,
    const MeshInfo & mi )
```

6.13.2.17 polyEval()

```
double polyEval (
    double x,
    double * coef,
    int degree )
```

6.13.2.18 polynDerMulti()

```
void polynDerMulti (
    int der,
    int max,
    int * multiplier )
```

6.14 util.h

[Go to the documentation of this file.](#)

```
00001 #ifndef UTIL_H_
00002 #define UTIL_H_
00003
00004 #include <vector>
00005 #include <utility>
00006 #include <set>
00007 #include <unordered_set>
00008 #include <array>
00009 #include <valarray>
00010 #include <algorithm>
00011 #include <numeric>
00012 #include <memory>
00013 #include <cstdlib>
00014 #include <type_traits>
00015 #include <iomanip>
00016
00017 #include "lapacke.h"
00018 #include "integral.h"
00019 #include <assert.h>
00020
00021 #include <petsc.h>
00022
00023 using vertex = valarray<double>;
00024 using indice = valarray<int>;
00025
00026 using vertexSet = vector<valarray<double>>;
00027 using indiceSet = vector<valarray<double>>;
00028
00029 template <typename T>
00030 void delete_pointed_to(T const ptr){
00031     delete ptr;
00032 }
00033
00034 /*
00035  *Containing essential information about mesh.
00036  */
00037 typedef struct {
00038
00039     // In the case, Cell and Vertex are maintained by same global size
00040     // They will still be stored separately for clarification.
00041
00042     int dim; // Total number of dimensions
00043
00044     vector<int> MPIlocalCellSize; // Local chunk size of cell without ghost layer
00045     vector<int> MPIlocalVertexSize; // Local chunk size of vertex without ghost layer
00046
00047     indice MPIlocalCellStart; // The starting cell index for MPI local part
00048     indice MPIlocalVertexStart; // The starting vertex index for MPI local part
00049
00050     vector<int> MPIglobalCellSize; // global chunk size of cell without ghost layer
00051     vector<int> MPIglobalVertexSize; // global chunk size of vertex without ghost layer
00052 }
```

```

00053     int cellGhostLayerSize; // size of ghost layer of cell
00054     int vertexGhostLayerSize; // size of ghost layer of node
00055
00056     vector<int> MPIlocalCellSizeFull; // Local chunk size of cells including ghost layer
00057     vector<int> MPIlocalVertexSizeFull; // Local chunk size of vertex including ghost layer
00058
00059     // Horizontal edges first than vertical edges
00060     int MPIglobalHoriEdgeSize;
00061     int MPIlocalHoriEdgeSize;
00062
00063     // Starting at 0 (has not stacked on horizontal edges yet)
00064     int MPIglobalVertEdgeSize;
00065     int MPIlocalVertEdgeSize;
00066
00067     // Containing all the local mesh vertex points here
00068     vertexSet lmesh;
00069
00070     double** localVals;
00071
00072     // Corner index within a single element
00073     const indiceSet edgeCorner {{0}, {1}};
00074     const indiceSet faceCorner {{0,0},{1,0},{1,1},{0,1}};
00075     const indiceSet volumeCorner {{0,0,0},{1,0,0},{1,1,0},{0,1,0},
00076                                   {0,0,1},{1,0,1},{1,1,1},{0,1,1}};
00077 } MeshInfo;
00078
00079 void AssignValuesMeshInfo(MeshInfo& mi, DM dmv, DM dmu);
00080
00081 // Generic auxiliary functions
00082 // Function return constant value
00083 double constFunc(valarray<double>& point, const vector<double>& param);
00084
00085 double constFunc(valarray<double>& point, const vector<double>& param, double c);
00086
00087 double constFunc();
00088
00089 double constFunc(double c);
00090
00091 // Functions calculate factorials
00092 int factorial(int top, int bottom);
00093
00094 int factorial(int top);
00095
00096 // Evaluation of polynomial using Horner's method
00097 double polyEval(double x, double * coef, int degree);
00098
00099 // Compute factorial coefficient for polynomial derivatives
00100 void polynDerMulti(int der, int max, int * multiplier);
00101
00102 // Local to global and global to local
00103 // All indices are referenced to cell indice
00104 indice MPIlocalToGlobal(indice local, const MeshInfo& mi);
00105 indice MPIGlobalToLocal(indice global, const MeshInfo& mi);
00106
00107 // Indice convention functions
00108 // Flatten indice into 1D array
00109 int FlatIndic(const MeshInfo& mi, int i, int j);
00110
00111 int FlatIndic(const int M, int i, int j);
00112 int FlatIndic(const MeshInfo& mi, const indice& p);
00113 int FlatIndic(const int M, const indice& p);
00114
00115 // Reverse process of flatten indices
00116 indice Bend(const MeshInfo& mi, int flat);
00117
00118 indice Bend(const int M, int flat);
00119
00120 #endif

```


Index

/home/renpo/Research/MantleSolver/src/mlweno/mlreconstruction.cpp, 31
/home/renpo/Research/MantleSolver/src/mlweno/mlreconstruction.h, 31, 32
/home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp, 32
/home/renpo/Research/MantleSolver/src/mlweno/polynomial.h, 33
/home/renpo/Research/MantleSolver/src/mlweno/slireconstruction.cpp, 35
/home/renpo/Research/MantleSolver/src/mlweno/slireconstruction.h, 35
/home/renpo/Research/MantleSolver/src/mlweno/stencil.h, 36, 37
/home/renpo/Research/MantleSolver/src/mlweno/util.cpp, 38
/home/renpo/Research/MantleSolver/src/mlweno/util.h, 41, 46
~basisPolynomial
 MLWENO::basisPolynomial, 10
~multiLevelReconstruction
 MLWENO::multiLevelReconstruction, 17
~singleLevelReconstruction
 MLWENO::singleLevelReconstruction, 20
~stencil
 MLWENO::stencil< T >, 23
~stencilPolynomial
 MLWENO::stencilPolynomial, 27
AddLevel
 MLWENO::multiLevelReconstruction, 17
AddReconstMethod
 MLWENO::multiLevelReconstruction, 17, 18
AssignValuesMeshInfo
 util.cpp, 38
 util.h, 43
basePoly
 polynomial.cpp, 33
basisPolynomial
 MLWENO::basisPolynomial, 9, 10
Bend
 util.cpp, 38, 39
 util.h, 43
CalculateSmoothnessIndic
 MLWENO::singleLevelReconstruction, 20
cellCentered
 MLWENO, 8
cellGhostLayerSize
 MeshInfo, 13
 CheckExist
 MLWENO::singleLevelReconstruction, 20
 CheckStencilPolynomials
 MLWENO::singleLevelReconstruction, 21
 CheckStencils
 MLWENO::singleLevelReconstruction, 21
 Clear
 MLWENO::multiLevelReconstruction, 18
 clearPtr
 MLWENO::stencil< T >, 23
 constFunc
 util.cpp, 39
 util.h, 43, 44
 CreateStencil
 MLWENO::stencil< T >, 24
 CreateStencilPolynomials
 MLWENO::singleLevelReconstruction, 21
delete_pointed_to
 util.h, 44
der
 MLWENO::basisPolynomial, 10
dim
 MeshInfo, 13
edgeCentered
 MLWENO, 8
edgeCorner
 MeshInfo, 13
eval
 MLWENO::basisPolynomial, 10
 MLWENO::stencilPolynomial, 28
faceCorner
 MeshInfo, 13
factorial
 util.cpp, 39, 40
 util.h, 44
FlatIndic
 util.cpp, 40
 util.h, 44, 45
getCoef
 MLWENO::basisPolynomial, 11
getl
 MLWENO::basisPolynomial, 11
 MLWENO::stencil< T >, 24
GetInfo
 MLWENO::multiLevelReconstruction, 18

- getJ
 - MLWENO::basisPolynomial, 11
 - MLWENO::stencil< T >, 24
- getK
 - MLWENO::stencil< T >, 24
- GetOrderX
 - MLWENO::stencilPolynomial, 28
- GetOrderY
 - MLWENO::stencilPolynomial, 28
- GetScale
 - MLWENO::singleLevelReconstruction, 21
 - MLWENO::stencilPolynomial, 28
- getSize
 - MLWENO::basisPolynomial, 11
 - MLWENO::stencil< T >, 24
- GetSizeX
 - MLWENO::singleLevelReconstruction, 21
- GetSizeY
 - MLWENO::singleLevelReconstruction, 22
- GetSmoothIndic
 - MLWENO::stencilPolynomial, 28
- GetStencil
 - MLWENO::stencil< T >, 24
- indice
 - util.h, 42
- indiceSet
 - util.h, 42
- Imesh
 - MeshInfo, 13
- localVals
 - MeshInfo, 13
- MeshInfo, 12
 - cellGhostLayerSize, 13
 - dim, 13
 - edgeCorner, 13
 - faceCorner, 13
 - Imesh, 13
 - localVals, 13
 - MPIglobalCellSize, 13
 - MPIglobalHoriEdgeSize, 13
 - MPIglobalVertEdgeSize, 14
 - MPIglobalVertexSize, 14
 - MPIlocalCellSize, 14
 - MPIlocalCellSizeFull, 14
 - MPIlocalCellStart, 14
 - MPIlocalHoriEdgeSize, 14
 - MPIlocalVertEdgeSize, 14
 - MPIlocalVertexSize, 14
 - MPIlocalVertexSizeFull, 15
 - MPIlocalVertexStart, 15
 - vertexGhostLayerSize, 15
 - volumeCorner, 15
- MLWENO, 7
 - cellCentered, 8
 - edgeCentered, 8
 - stencilType, 7
 - vertexCentered, 8
- MLWENO::basisPolynomial, 9
 - ~basisPolynomial, 10
 - basisPolynomial, 9, 10
 - der, 10
 - eval, 10
 - getCoef, 11
 - getI, 11
 - getJ, 11
 - getSize, 11
 - operator(), 11
 - printCoef, 12
 - setCoef, 12
 - setMaxDegree, 12
- MLWENO::multiLevelReconstruction, 15
 - ~multiLevelReconstruction, 17
 - AddLevel, 17
 - AddReconstMethod, 17, 18
 - Clear, 18
 - GetInfo, 18
 - multiLevelReconstruction, 16
 - PrintNonLinearWgts, 18
 - PrintSmoothnessIndicator, 18
 - UpdateNonLinearWgts, 18
 - UpdateTwoStageNonLinearWgts, 19
- MLWENO::singleLevelReconstruction, 19
 - ~singleLevelReconstruction, 20
 - CalculateSmoothnessIndic, 20
 - CheckExist, 20
 - CheckStencilPolynomials, 21
 - CheckStencils, 21
 - CreateStencilPolynomials, 21
 - GetScale, 21
 - GetSizeX, 21
 - GetSizeY, 22
 - PrintSmoothnessIndicator, 22
 - singleLevelReconstruction, 20
- MLWENO::stencil< T >, 22
 - ~stencil, 23
 - clearPtr, 23
 - CreateStencil, 24
 - getI, 24
 - getJ, 24
 - getK, 24
 - getSize, 24
 - GetStencil, 24
 - operator(), 24, 25
 - setI, 25
 - setJ, 26
 - setK, 26
 - SetStencil, 26
 - stencil, 23
- MLWENO::stencilPolynomial, 27
 - ~stencilPolynomial, 27
 - eval, 28
 - GetOrderX, 28
 - GetOrderY, 28
 - GetScale, 28

- GetSmoothIndic, [28](#)
 - operator(), [28](#), [29](#)
 - printCoef, [29](#)
 - SetCollapsePolyn, [29](#)
 - SetStencilPolynomials, [29](#)
 - SetUpScale, [29](#), [30](#)
 - stencilPolynomial, [27](#)
- MPIGlobalCellSize
 - MeshInfo, [13](#)
- MPIGlobalHoriEdgeSize
 - MeshInfo, [13](#)
- MPIGlobalToLocal
 - util.cpp, [40](#)
 - util.h, [45](#)
- MPIGlobalVertEdgeSize
 - MeshInfo, [14](#)
- MPIGlobalVertexSize
 - MeshInfo, [14](#)
- MPIIlocalCellSize
 - MeshInfo, [14](#)
- MPIIlocalCellSizeFull
 - MeshInfo, [14](#)
- MPIIlocalCellStart
 - MeshInfo, [14](#)
- MPIIlocalHoriEdgeSize
 - MeshInfo, [14](#)
- MPILocalToGlobal
 - util.cpp, [41](#)
 - util.h, [45](#)
- MPIIlocalVertEdgeSize
 - MeshInfo, [14](#)
- MPIIlocalVertexSize
 - MeshInfo, [14](#)
- MPIIlocalVertexSizeFull
 - MeshInfo, [15](#)
- MPIIlocalVertexStart
 - MeshInfo, [15](#)
- multiLevelReconstruction
 - MLWENO::multiLevelReconstruction, [16](#)
- operator()
 - MLWENO::basisPolynomial, [11](#)
 - MLWENO::stencil< T >, [24](#), [25](#)
 - MLWENO::stencilPolynomial, [28](#), [29](#)
- polyEval
 - util.cpp, [41](#)
 - util.h, [45](#)
- polynDerMulti
 - util.cpp, [41](#)
 - util.h, [46](#)
- polynomial.cpp
 - basePoly, [33](#)
- printCoef
 - MLWENO::basisPolynomial, [12](#)
 - MLWENO::stencilPolynomial, [29](#)
- PrintNonLinearWgts
 - MLWENO::multiLevelReconstruction, [18](#)
- PrintSmoothnessIndicator
 - MLWENO::multiLevelReconstruction, [18](#)
 - MLWENO::singleLevelReconstruction, [22](#)
- setCoef
 - MLWENO::basisPolynomial, [12](#)
- SetCollapsePolyn
 - MLWENO::stencilPolynomial, [29](#)
- setI
 - MLWENO::stencil< T >, [25](#)
- setJ
 - MLWENO::stencil< T >, [26](#)
- setK
 - MLWENO::stencil< T >, [26](#)
- setMaxDegree
 - MLWENO::basisPolynomial, [12](#)
- SetStencil
 - MLWENO::stencil< T >, [26](#)
- SetStencilPolynomials
 - MLWENO::stencilPolynomial, [29](#)
- SetUpScale
 - MLWENO::stencilPolynomial, [29](#), [30](#)
- singleLevelReconstruction
 - MLWENO::singleLevelReconstruction, [20](#)
- stencil
 - MLWENO::stencil< T >, [23](#)
- stencilPolynomial
 - MLWENO::stencilPolynomial, [27](#)
- stencilType
 - MLWENO, [7](#)
- UpdateNonLinearWgts
 - MLWENO::multiLevelReconstruction, [18](#)
- UpdateTwoStageNonLinearWgts
 - MLWENO::multiLevelReconstruction, [19](#)
- util.cpp
 - AssignValuesMeshInfo, [38](#)
 - Bend, [38](#), [39](#)
 - constFunc, [39](#)
 - factorial, [39](#), [40](#)
 - FlatIndic, [40](#)
 - MPIGlobalToLocal, [40](#)
 - MPILocalToGlobal, [41](#)
 - polyEval, [41](#)
 - polynDerMulti, [41](#)
- util.h
 - AssignValuesMeshInfo, [43](#)
 - Bend, [43](#)
 - constFunc, [43](#), [44](#)
 - delete_pointed_to, [44](#)
 - factorial, [44](#)
 - FlatIndic, [44](#), [45](#)
 - indice, [42](#)
 - indiceSet, [42](#)
 - MPIGlobalToLocal, [45](#)
 - MPILocalToGlobal, [45](#)
 - polyEval, [45](#)
 - polynDerMulti, [46](#)
 - vertex, [42](#)
 - vertexSet, [43](#)

- vertex
 - util.h, [42](#)
- vertexCentered
 - MLWENO, [8](#)
- vertexGhostLayerSize
 - MeshInfo, [15](#)
- vertexSet
 - util.h, [43](#)
- volumeCorner
 - MeshInfo, [15](#)